Get unlimited access to the best of Medium for less than $1/week. Become a member

Deep Technical Guide to LangChain: Building Modular LLM Applications

3 min read · Just now

Rncs Shreyavinodkumar

Listen Share More

1. Introduction to LangChain

What is LangChain?

LangChain is an open-source framework designed to simplify the development of applications powered by Large Language Models (LLMs). It provides structured abstractions that allow developers to build complex AI workflows by chaining together multiple components such as prompts, models, tools, and memory.

At its core, LangChain transforms LLM usage from simple API calls into **modular, composable pipelines.**

Why is LangChain Important?

Modern LLM applications are no longer just “input → output” systems. They require:

- Context awareness

- External data integration
- Multi-step reasoning
- Tool usage (APIs, databases)

LangChain solves these by introducing **structured orchestration**.

Problems It Solves

ProblemSolution by LangChainRepetitive prompt handlingPrompt templatesStateless responsesMemoryMulti-step reasoningChainsExternal integrationsTools & AgentsData retrievalVector stores

2. Core Components of LangChain

2.1 LLMs and Chat Models

Concept

LLMs are the core engines that generate responses. Chat models are conversational variants.

Why it Exists

To abstract API differences between providers like OpenAI, HuggingFace, etc.

Code Example

```
from langchain_openai import ChatOpenAI
```

```
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

response = llm.invoke("Explain LangChain in simple terms")
print(response.content)
```

2.2 Prompts and Prompt Templates

Concept

Prompt templates allow dynamic input injection.

Why it Exists

Avoid hardcoding prompts and enable reusable structures.

Code Example

```
from langchain.prompts import PromptTemplate
```

```
template = PromptTemplate(
    input_variables=["topic"],
    template="Explain {topic} in simple terms"
)

prompt = template.format(topic="LangChain")
print(prompt)
```

2.3 Chains

Concept

Chains combine multiple steps into a pipeline.

Why it Exists

To automate multi-step LLM workflows.

Code Example

```
from langchain.chains import LLMChain
from langchain_openai import ChatOpenAI
from langchain.prompts import PromptTemplate
```

```
llm = ChatOpenAI()

prompt = PromptTemplate(
    input_variables=["topic"],
    template="Write a short explanation about {topic}"
)

chain = LLMChain(llm=llm, prompt=prompt)

output = chain.run("LangChain")
print(output)
```

2.4 Memory

Concept

Memory allows the system to remember past interactions.

Why it Exists

LLMs are stateless by default.

Code Example

```
from langchain.memory import ConversationBufferMemory
from langchain.chains import ConversationChain
from langchain_openai import ChatOpenAI
```

```
memory = ConversationBufferMemory()
conversation = ConversationChain(
    llm=ChatOpenAI(),
    memory=memory
)

print(conversation.run("Hi, my name is Alex"))
print(conversation.run("What is my name?"))
```

2.5 Agents

Concept

Agents decide what actions to take dynamically.

Why it Exists

For reasoning + decision-making beyond static chains.

2.6 Tools

Concept

Tools are external functions APIs agents can use.

Code Example

```
from langchain.tools import tool
```

```
@tool
def multiply(a: int, b: int) -> int:
    return a * b
```

2.7 Agent with Tool Example

```
from langchain.agents import initialize_agent
from langchain.agents.agent_types import AgentType
from langchain_openai import ChatOpenAI
```

```
llm = ChatOpenAI()

tools = [multiply]

agent = initialize_agent(
    tools,
    llm,
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
    verbose=True
)

agent.run("What is 5 times 6?")
```

2.8 Document Loaders

Concept

Load external data (PDF, text, web).

Code Example

```
from langchain.document_loaders import TextLoader
```

```
loader = TextLoader("data.txt")
docs = loader.load()
```

2.9 Vector Stores / Indexes

Concept

Store embeddings for semantic search.

Why it Exists

Enable Retrieval-Augmented Generation (RAG).

Code Example

```
from langchain.vectorstores import FAISS
from langchain.embeddings import OpenAIEmbeddings
```

```
embeddings = OpenAIEmbeddings()
db = FAISS.from_documents(docs, embeddings)

query = "What is this document about?"
results = db.similarity_search(query)

print(results[0].page_content)
```

3. Architecture Explanation

Flow Overview

```
User Input
   ↓
Prompt Template
   ↓
LLM
   ↓
Chain
   ↓
Agent / Tools (optional)
   ↓
Final Output
```

Explanation

1. User provides input
2. Prompt formats the input
3. LLM processes it
4. Chain organizes multi-step flow
5. Agent decides actions
6. Tools execute tasks
7. Output returned

4. Hands-on Code Examples

4.1 Basic LLM Call

```
llm.invoke("What is AI?")
```

4.2 Prompt Template Usage

```
template.format(topic="Machine Learning")
```

4.3 Simple Chain

```
chain.run("Neural Networks")
```

4.4 Agent with Tool

```
agent.run("Multiply 8 and 7")
```

4.5 Memory Example

```
conversation.run("Remember I like Python")
conversation.run("What do I like?")
```

5. Real-World Use Cases

Use Case 1: Chatbot with Memory

Problem: Chatbots forget context

Solution: Use ConversationBufferMemory

Components: LLM + Memory + Chain

Use Case 2: Document QA System

Problem: Extract answers from documents
Solution: RAG using Vector Store
Components: Loader + Embeddings + Retriever + LLM

Use Case 3: AI Assistant with Tools

Problem: Perform real-world actions
Solution: Agent with Tools
Components: Agent + Tool + LLM

6. Advantages and Limitations

Advantages

- Modular architecture
- Easy integration
- Rapid prototyping
- Supports multiple LLM providers

Limitations

- High latency in complex chains
- Debugging is difficult
- Cost increases with API calls
- Overkill for simple tasks

When NOT to Use LangChain

- Simple one-shot prompts
- Low-latency systems
- Cost-sensitive applications

7. Conclusion

LangChain provides a powerful abstraction layer for building advanced LLM applications. It shifts development from raw API calls to structured, modular pipelines.

Key Takeaways

- LangChain enables composability
- Agents introduce reasoning capability
- Memory enables context-aware systems
- Vector stores power RAG systems

Future Scope

- LangGraph for multi-agent workflows
- Autonomous AI systems
- Production-grade LLM orchestration

This framework is essential for building real-world GenAI systems and mastering it is a critical step toward advanced AI engineering.

Genai

Edit profile

Written by Rncs Shreyavinodkumar

0 followers · 1 following

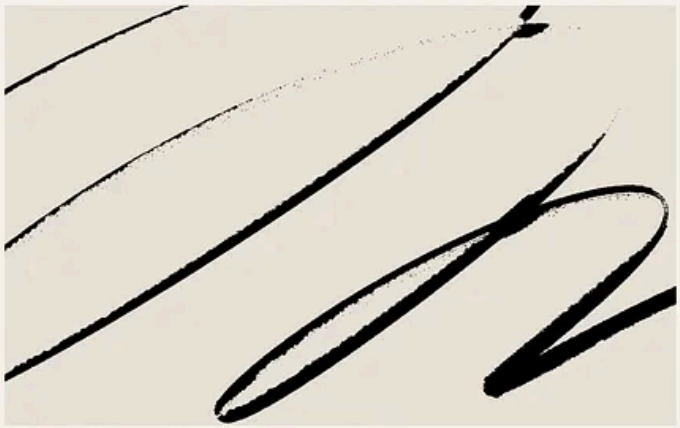
No responses yet

S Rncs Shreyavinodkumar

What are your thoughts?

Open in app ↗

Medium



Rncs Shreyavinodkumar

From Raw Text to Machine Learning: A Complete NLP Pipeline Explained

Introduction to Natural Language Processing (NLP)

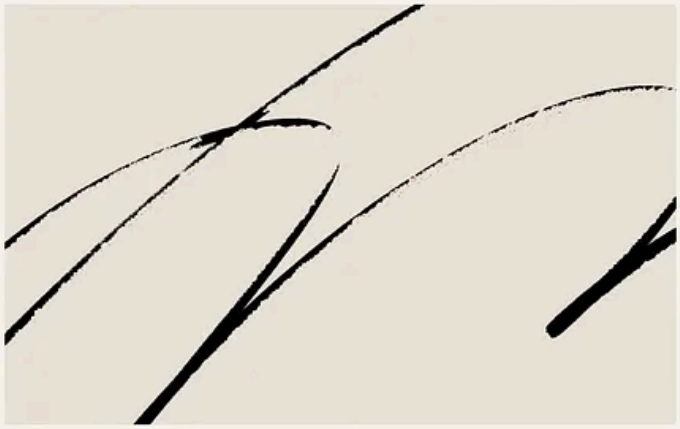
Mar 30

Rncs Shreyavinodkumar

Understanding Token Classification in NLP: From Words to Meaning

Natural Language Processing (NLP) is one of the most exciting areas in artificial intelligence. It enables machines to understand...

Apr 4



Rncs Shreyavinodkumar

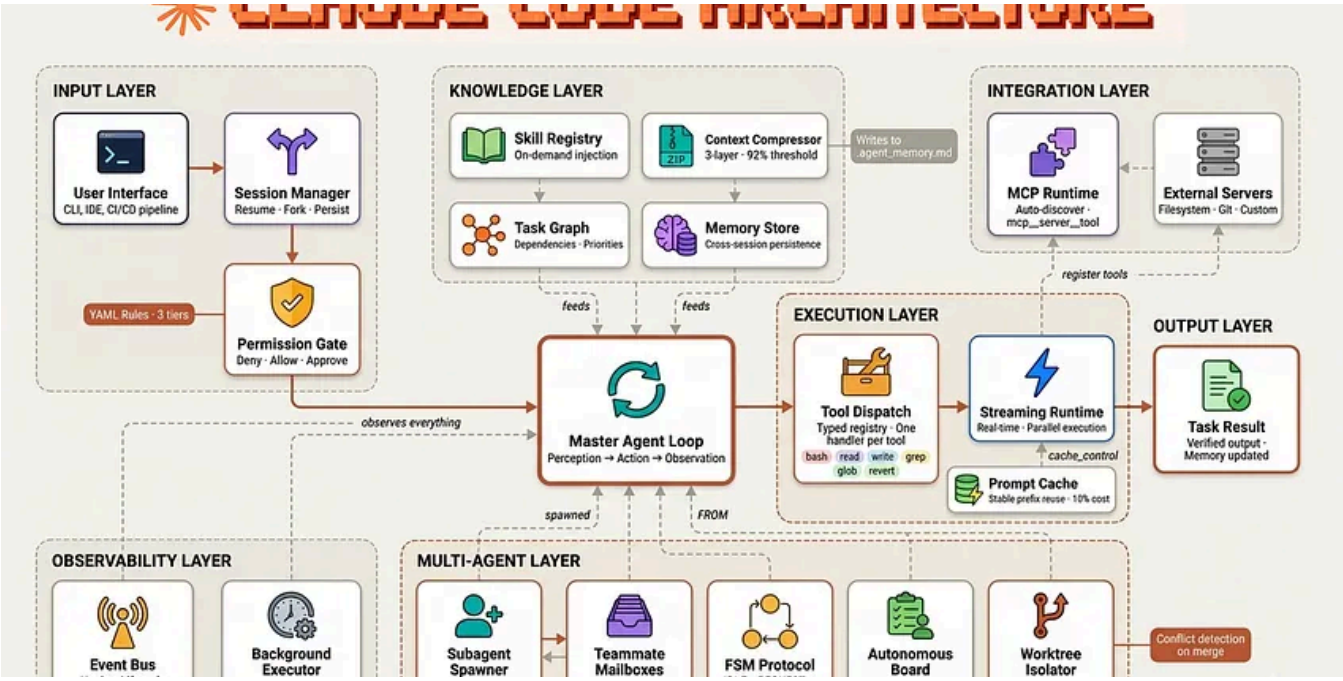
Lists vs Tuples in Python: When Should You Use Which?

Lists vs Tuples in Python: When Should You Use Which?

Feb 16

See all from Rncs Shreyavinodkumar

Recommended from Medium

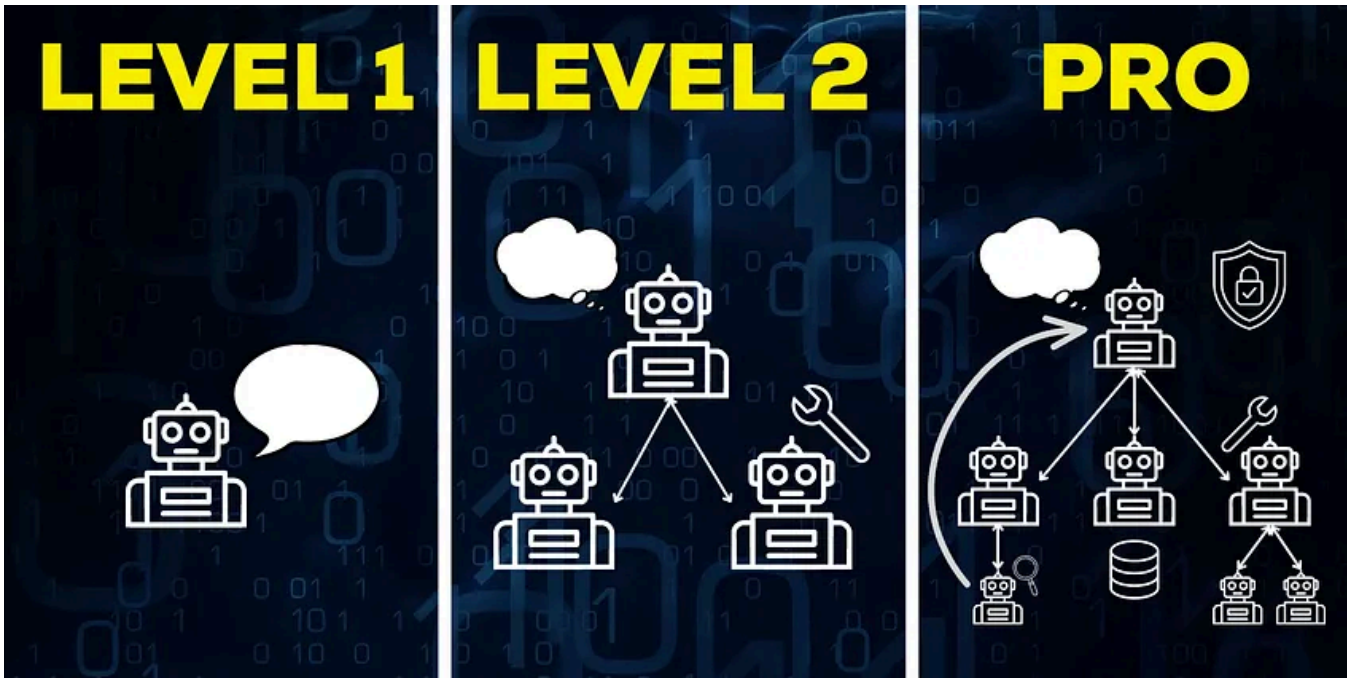


In Level Up Coding by Fareed Khan

Building Claude Code with Harness Engineering

Multi-agents, MCP, skills system, context pipelines and more

6d ago 1K 8



In Data Science Collective by Marina Wyss

AI Agents: Complete Course

From beginner to intermediate to production.

Dec 6, 2025 5.7K 231

Joe Njenga

Claude Code Ultraplan Launched: I Just Tested It (And It's Better Than It Looks)

Anthropic has added Claude Code ultraplan, and I was quick to test it. You might like it or hate it for one reason — I'll talk about that...

Apr 5 742 23

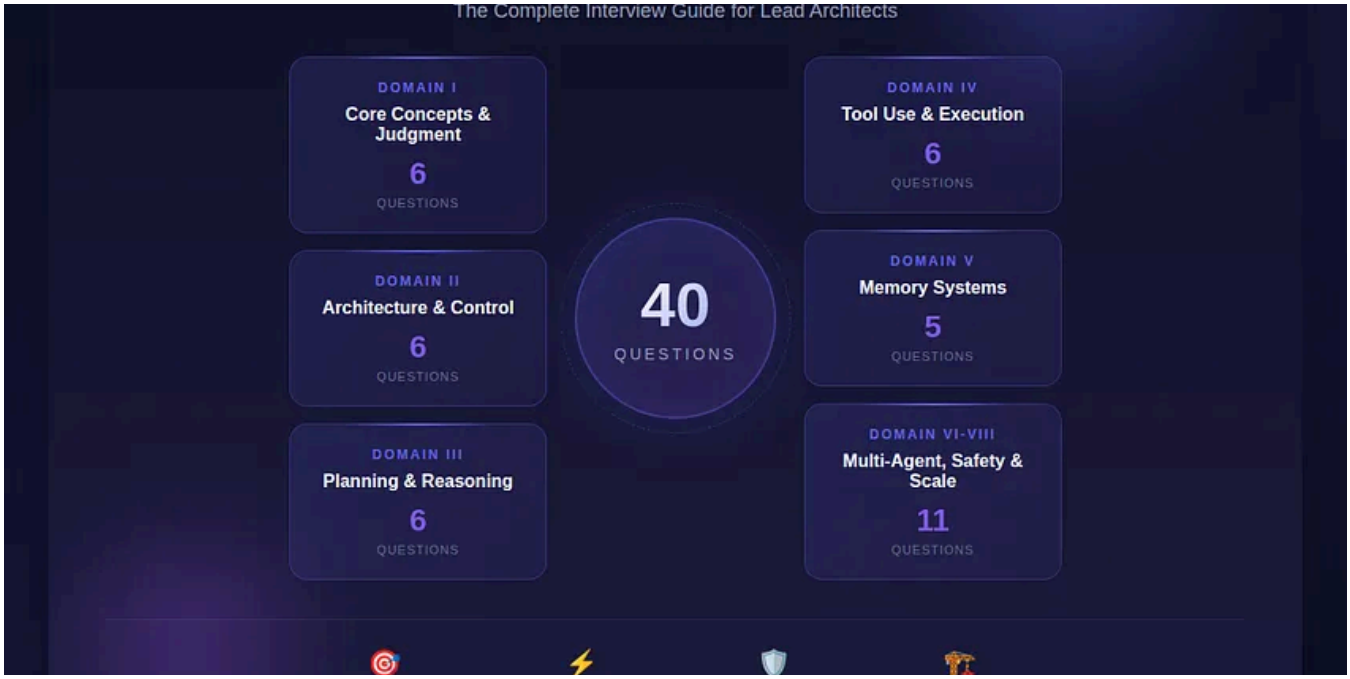
E2B	2.3B	4GB	85	6.1/10	6.5/10	On-device, edge
E4B	4.5B	8GB	38	7.4/10	7.8/10	Local dev, laptops
26B MoE	**3.8B active**	**18GB**	**42**	**9.1/10**	**8.9/10**	**Most use cases**

In Towards AI by Chew Loong Nian

I Tested All 4 Gemma 4 Models: The 26B One Is Cheating (In the Best Way)

Google dropped Gemma 4 this week under Apache 2.0 — free to use, free to commercialize, no strings attached. Four model sizes landed at...

Apr 4 145 1



TechEon

The Complete Agentic AI System Design Interview Guide 2026

A senior engineer's handbook for navigating the toughest agentic AI interviews

Jan 29 107 2

unicodeveloper

10 Must-Have Skills for Claude (and Any Coding Agent) in 2026

The definitive guide to agent skills that change how Claude Code, Cursor, Gemini CLI, and other AI coding assistants perform in production.

Mar 9 1.3K 21

See more recommendations